Java OOP is actually pretty simple. I am adding my own style and quickly explaining my understanding.
First one is Class
- This class is something I define first.
For example: Car(has color, wheels, doors, engine...)
The things inside the parentheses are the parts that the real world object should have.
Second one is Object
- This object is made from the class template, but now it has real values.
For example: Car(
- color: red
- wheels: round
- doors: two doors
- engine: V8
)
The properties inside here are created from the class template.
Done with the basic idea. Using an actual coding example is much easier to explain.
Now we think about what a vending machine should have.

The picture above shows what properties and functions this vending machine should have.
We used new to create an actual object, and then used vm to reference it.
The int, String, char, and boolean that we already know are all data types.
When we create a class, that class also becomes our own data type.
- VendingMachine is my class.
- vm is my variable name.
- new VendingMachine() creates a real vending machine object, and then vm references it.
Methods and member variables
Methods are called through objects.
variableName.methodName();
For example:

```
vm.showBalance();
```

This means Java uses vm to find that VendingMachine object, then runs the method on that object.
Inside the method, this means the current object.
This is my current practice code idea
This is my first class.
First, I created the VendingMachine template.
Second, I defined member variables:

```
float balance;
float total;
ArrayList<Item> items = new ArrayList<>();
```

balance means the money currently inserted.
total means the total money collected by the machine.
items is an ArrayList that stores Item objects.
Third, I declared some methods.
For example, showPrompt(), insertmoney(), showBalance(), and getfood().
Fourth, I wrote the getfood() method. I mainly want to explain this method.
This method first gets one item from the item list:

```
Item item = this.items.get(0);
```

Then it compares the balance and the item price:

```
if (this.balance >= item.price)
```

If the balance is enough, I get the item. Then the machine subtracts the item price from balance and adds that price to total.
Fifth, in main, I created two Items:

```
Item soda = new Item(1, 2.5f, "Coca Cola");
Item water = new Item(2, 1.0f, "Water");
```

Then I created a vm:

```
VendingMachine vm = new VendingMachine();
```

vm points to a VendingMachine object.
More accurately, vm does not point to items directly.
vm points to the VendingMachine object, and items is a member variable inside that object.
Then I use vm to reach items:

```
vm.items.add(soda);
vm.items.add(water);
```

items is from Java standard library ArrayList.
items.add(soda) calls the add() method and puts the Item object referenced by soda into the list.
So I should not say it adds a variable. It adds an Item object reference into the ArrayList.
If I want to show all items, I can loop through the list:

```
for (int i = 0; i < items.size(); i++) {
    Item item = items.get(i);
    System.out.println(item.name + " " + item.price);
}
```

My code screenshot:

```
1  // Source code is decompiled from a .class file using FernFlower decompiler (from I
2  import java.util.ArrayList;
3
4  public class VendingMachine {
5     float banlance;
6     float total;
7     ArrayList<Item> items = new ArrayList();
8
9     public VendingMachine() {
10    }
11
12    void showPrompt() {
13       System.out.println("Welcome to the Vending Machine!");
14       System.out.println("Please insert coins.");
15    }
16
17    void insertmoney(float amount) {
18       this.banlance += amount;
19    }
20
21    void showBalance() {
22       System.out.println("Balance: " + this.banlance);
23    }
24
25    void getfood() {
26       Item items = (Item)this.items.get(0);
27       if (this.banlance >= items.price) {
28          System.out.println("Enjoy your item!" + items.name);
29          this.banlance -= items.price;
30          this.total += items.price;
31       } else {
32          System.out.println("Insufficient balance. Please insert more coins.");
33       }
34
35    }
36
37    public static void main(String[] args) throws Exception {
38       Item soda = new Item(1, 2.5F, "Coca Cola");
39       Item Water = new Item(2, 1.0F, "Water");
40       VendingMachine vm = new VendingMachine();
41       vm.items.add(soda);
42       vm.items.add(Water);
43    }
44 }
45
```

My understanding:
Item describes what a product has.

VendingMachine manages money and stores many Item objects.
main creates the objects and connects them together.